Markdown to PDF Converter

A simple Python utility to convert a Markdown (`.md`) file into a professionally formatted PDF document.

The application uses:

- **Python**
- **Markdown** for converting Markdown to HTML
- **WeasyPrint** for converting HTML to PDF

Features

- Convert `.md` files to `.pdf`
- Supports Markdown headings
- Supports paragraphs
- Supports ordered and unordered lists
- Supports tables
- Supports fenced code blocks
- Supports inline code
- Supports blockquotes
- Supports Markdown images
- Supports links
- A4 PDF output
- Custom PDF styling using CSS
- Relative image paths are supported

Project Structure

```
markdown-to-pdf/
│
├── venv/
│
├── md_to_pdf.py
```

```
├── README.md
└── requirements.txt
```

Requirements

Make sure the following are installed:

- Python 3.9+
- pip

Check Python version:

```
python --version
```

Check pip:

```
pip --version
```

Create Project

Create the project directory:

```
mkdir markdown-to-pdf
cd markdown-to-pdf
```

Create Virtual Environment

Windows

```
python -m venv venv
```

Activate the virtual environment:

```
.\venv\Scripts\Activate.ps1
```

If PowerShell execution policy prevents activation:

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
```

Then activate:

```
.\venv\Scripts\Activate.ps1
```

Linux / macOS

```
python3 -m venv venv
source venv/bin/activate
```

Install Dependencies

```
pip install markdown weasyprint
```

Or install everything from `requirements.txt`:

```
pip install -r requirements.txt
```

Generate requirements.txt

After installing the dependencies:

```
pip freeze > requirements.txt
```

Example:

```
Markdown==...
weasyprint==...
```

Application Code

The main application is:

```
md_to_pdf.py
```

The application accepts a Markdown file as input and generates a PDF file.

Usage

Basic Usage

```
python md_to_pdf.py README.md
```

This generates:

```
README.pdf
```

Specify Output PDF

```
python md_to_pdf.py README.md documentation.pdf
```

Result:

```
documentation.pdf
```

Example

Suppose the project contains:

```
markdown-to-pdf/
│
├── md_to_pdf.py
├── sample.md
└── requirements.txt
```

Run:

```
python md_to_pdf.py sample.md
```

The generated file will be:

```
markdown-to-pdf/
│
├── md_to_pdf.py
├── sample.md
├── sample.pdf
└── requirements.txt
```

Markdown Example

Create a file named:

```
sample.md
```

with the following content:

````
# Sample Document

This is a sample Markdown document.

## Features

- Feature 1
- Feature 2
- Feature 3

## Code Example

```java
public class HelloWorld {

    public static void main(String[] args) {
        System.out.println("Hello World");
    }
}
````

Table

Name	Age	City
John	30	Delhi
````

Name	Age	City
Mike	28	Mumbai
David	35	Pune

Quote

> This is a sample blockquote.

Image

Sample Image

````
Generate the PDF:

```bash
python md_to_pdf.py sample.md
````

Images

Images referenced from Markdown are resolved relative to the Markdown file.

Example:

```
![Architecture](images/architecture.png)
```

Project structure:

```
markdown-to-pdf/
│
├── md_to_pdf.py
├── sample.md
│
└── images/
    └── architecture.png
```

Run:
```

```
python md_to_pdf.py sample.md
```

The image will be included in the generated PDF.

Supported Markdown

The application enables the following Markdown extensions:

```
extensions=[
    "extra",
    "tables",
    "fenced_code",
    "toc",
    "sane_lists"
]
```

Headings

```
# Heading 1

## Heading 2

### Heading 3
```

Bold and Italic

```
**Bold**

*Italic*

***Bold Italic***
```

Lists

```
- Item 1
- Item 2
- Item 3
```

Ordered list:

```
1. First
2. Second
3. Third
```

Code

Inline code:

```
Use `System.out.println()` to print output.
```

Code block:

````
```java
System.out.println("Hello");
```
````

Tables

```
| Name | Age |
|------|-----|
| Prem | 30 |
| John | 35 |
```

Links

```
[Python](https://www.python.org)
```

Images

```
![Image](images/example.png)
```

Blockquote

```
> This is a blockquote.
```

PDF Styling

The generated PDF uses CSS for formatting.

The PDF is configured for:

```
 Page Size : A4
Margins    : 20mm / 18mm
Font       : Arial
```

The styling includes:

- Heading formatting
- Paragraph spacing
- Code block formatting
- Table borders
- Table headers
- Blockquotes
- Image scaling
- Link styling
- Lists

Error Handling

If the Markdown file does not exist:

```
python md_to_pdf.py missing.md
```

The application reports:

```
FileNotFoundError: Markdown file not found
```

Command-Line Examples

Convert Markdown using the default PDF name:

```
python md_to_pdf.py README.md
```

Convert to a custom PDF name:

```
python md_to_pdf.py README.md output.pdf
```

Convert another Markdown file:

```
python md_to_pdf.py documentation.md documentation.pdf
```

Deactivate Virtual Environment

After completing the work:

```
deactivate
```

Recreate Environment

If the virtual environment is deleted, recreate it using:

```
python -m venv venv
```

Windows:

```
.\venv\Scripts\Activate.ps1
```

Linux/macOS:

```
source venv/bin/activate
```

Install dependencies:

```
pip install -r requirements.txt
```

Troubleshooting

Python command not found

Check whether Python is installed:

```
python --version
```

On some systems:

```
python3 --version
```

WeasyPrint Import Error

Test the installation:

```
python -c "import weasyprint; print(weasyprint.__version__)"
```

If an error occurs, reinstall:

```
pip uninstall weasyprint -y
pip install weasyprint
```

Markdown Import Error

Test:

```
python -c "import markdown; print(markdown.__version__)"
```

If necessary:

```
pip install markdown
```

Complete Setup

For a fresh Windows installation:

```
mkdir markdown-to-pdf
cd markdown-to-pdf

python -m venv venv

.\venv\Scripts\Activate.ps1

python -m pip install --upgrade pip

pip install markdown weasyprint

pip freeze > requirements.txt
```

Then create:

```
md_to_pdf.py
```

and:

```
README.md
```

Run:

```
python md_to_pdf.py README.md
```

Future Enhancements

Possible improvements include:

- Syntax highlighting for Java, Python, JavaScript, JSON, XML, etc.
- Automatic table of contents
- Page numbers
- PDF header and footer
- Custom fonts
- Custom themes
- Cover page
- Automatic PDF metadata
- Clickable table of contents
- Better code block styling
- Support for Mermaid diagrams

- Support for PlantUML diagrams
- Command-line options for page size and margins

License

This project is provided for learning and personal use.

You are free to modify and extend the application according to your requirements.